# [Dat Nguyen] Userspace Debugging Toolkit: GDB, strace & Valgrind

## Userspace Debugging Toolkit: GDB, strace & Valgrind

### 1. GDB

GDB is the most important to debug with embedded developers.

GDB help us interact inside the program when it running. That mean we can stop the program wherever we want, display the value of variables, register and show the change of them.

**Breakpoints, Watchpoints, Stepping**

- Breakpoint: Stops the program at a specific line of code or function.
- Watchpoint: Stops the program whenever the value of a variable or memory location changes (very useful for finding which code modified a global variable).
- Stepping:
  - step (s): Executes line-by-line, stepping into functions if called.
  - next (n): Executes line-by-line, stepping over functions.
  - continue (c): Resumes execution until the next Breakpoint or Watchpoint.
  - finish: Runs until the current function returns, then stops.

**Inspecting Variables & Memory**

Used to check variable values or inspect Heap/Stack memory:

- p: Prints the value of a variable or C expression.
- display : Automatically prints the variable value every time the program stops.
- x/format address

**Stack Frames & Core Dumps**

When a program crashes (Segmentation Fault), the Stack Trace shows the exact sequence of function calls leading to the crash.

- backtrace (bt): Displays the list of Stack Frames from the current location back to main().
- frame < number> : Switches GDB context to a specific frame to inspect local variables at that point (info locals).

**Multi-thread Debugging (Thread Debugging & thread apply)**

When an application has multiple POSIX threads running at the same time, GDB manages each thread separately:

- info threads: Lists all running threads and their GDB IDs.
- thread < id>: Switches GDB control to the specified thread.
- thread apply <ids|all> < command>: Runs a command on multiple threads at once.

**Fork Debugging (Multi-process Debugging)**

By default, when encountering a fork() call, GDB follows the parent process and lets the child process run freely. Change this behavior with these settings:

- set follow-fork-mode <parent | child>: Chooses which process GDB will follow after fork().
- set detach-on-fork <on | off>:
  - on (Default): The other process runs independently.
  - off: GDB keeps both processes under control, allowing you to switch between Parent and Child using the "inferior" command.

```
int global_counter = 0;

void* thread_func(void* arg) {
    int id = *(int*)arg;
    for (int i = 0; i < 3; i++) {
        global_counter++;
        printf("[Thread %d] Counter: %d\n", id, global_counter);
        sleep(1);
    }
    return NULL;
}

void cause_crash(int *ptr) {
    printf("Writing to pointer...\n");
    *ptr = 42; // Segmentation Fault if ptr == NULL
}
```

```
void do_fork_demo() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("[Child Process] PID: %d\n", getpid());
        exit(0);
    } else {
        printf("[Parent Process] Spawned child PID: %d\n", pid);
        wait(NULL);
    }
}

int main() {
    printf("=== GDB Playground Starting ===\n");

    // 1. Thread Demo
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;
    pthread_create(&t1, NULL, thread_func, &id1);
    pthread_create(&t2, NULL, thread_func, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    // 2. Fork Demo
    do_fork_demo();

    // 3. Crash Demo (test Core Dump)
    int *bad_ptr = NULL;
    cause_crash(bad_ptr);

    return 0;
}
```

**Output**

gcc -g -O0 -pthread test.c -o app

gdb -q ./app

```
(gdb) b main
Breakpoint 1 at 0x1371: file test.c, line 37.
(gdb) b thread_func
Breakpoint 2 at 0x1279: file test.c, line 12.
(gdb) watch global_counter
Hardware watchpoint 3: global_counter
(gdb) set follow-fork-mode parent
(gdb) break do_fork_demo
Breakpoint 4 at 0x130c: file test.c, line 27.
(gdb) run

(gdb) continue
Continuing.
=== GDB Playground Starting ===
[New Thread 0x7ffff7bff6c0 (LWP 26493)]
[Switching to Thread 0x7ffff7bff6c0 (LWP 26493)]
Thread 2 "app" hit Breakpoint 2, thread_func (arg=0x7fffffffd628) at test.c:12
12          int id = *(int*)arg;

(gdb) next
[New Thread 0x7ffff73fe6c0 (LWP 26494)]
[Switching to Thread 0x7ffff73fe6c0 (LWP 26494)]
Thread 3 "app" hit Breakpoint 2, thread_func (arg=0x7fffffffd62c) at test.c:12
12          int id = *(int*)arg;

(gdb) next
[Switching to Thread 0x7ffff7bff6c0 (LWP 26493)]
Thread 2 "app" hit Hardware watchpoint 3: global_counter
Old value = 0
New value = 1
thread_func (arg=0x7fffffffd628) at test.c:15
15              printf("[Thread %d] Counter: %d\n", id, global_counter);
(gdb) info local
i = 0
id = 1
(gdb) p global_counter
$1 = 1

(gdb) next
[Switching to Thread 0x7ffff73fe6c0 (LWP 26494)]
```

```
Thread 3 "app" hit Hardware watchpoint 3: global_counter
Old value = 1
New value = 2
thread_func (arg=0x7fffffffd62c) at test.c:15
15              printf("[Thread %d] Counter: %d\n", id, global_counter);
(gdb) info local
i = 0
id = 2
(gdb) p global_counter
$2 = 2

(gdb) next
[Thread 1] Counter: 1
[Thread 2] Counter: 2
16              sleep(1);

(gdb) next
[Switching to Thread 0x7ffff7bff6c0 (LWP 26493)]
Thread 2 "app" hit Hardware watchpoint 3: global_counter
Old value = 2
New value = 3
thread_func (arg=0x7fffffffd628) at test.c:15
15              printf("[Thread %d] Counter: %d\n", id, global_counter);
(gdb) info local
i = 1
id = 1
(gdb) p global_counter
$3 = 3

(gdb) next
[Thread 1] Counter: 3
16              sleep(1);
(gdb) next
[Switching to Thread 0x7ffff73fe6c0 (LWP 26494)]
Thread 3 "app" hit Hardware watchpoint 3: global_counter
Old value = 3
New value = 4
thread_func (arg=0x7fffffffd62c) at test.c:15
15              printf("[Thread %d] Counter: %d\n", id, global_counter);
(gdb) info local
```

```
i = 1
id = 2
(gdb) p global_counter
$4 = 4

(gdb) next
[Thread 2] Counter: 4
16              sleep(1);
(gdb) info local
i = 1
id = 2
(gdb) p global_counter
$5 = 4

(gdb) next
[Switching to Thread 0x7ffff7bff6c0 (LWP 26493)]
Thread 2 "app" hit Hardware watchpoint 3: global_counter
Old value = 4
New value = 5
thread_func (arg=0x7fffffffd628) at test.c:15
15              printf("[Thread %d] Counter: %d\n", id, global_counter);
(gdb) info local
i = 2
id = 1
(gdb) p global_counter
$6 = 5

(gdb) next
[Thread 1] Counter: 5
16              sleep(1);
(gdb) next
[Switching to Thread 0x7ffff73fe6c0 (LWP 26494)]
Thread 3 "app" hit Hardware watchpoint 3: global_counter
Old value = 5
New value = 6
thread_func (arg=0x7fffffffd62c) at test.c:15
15              printf("[Thread %d] Counter: %d\n", id, global_counter);

(gdb)
Continuing.
```

```
[Thread 2] Counter: 6
[Thread 0x7ffff7bff6c0 (LWP 33966) exited]
[Thread 0x7ffff73fe6c0 (LWP 33967) exited]
[Switching to Thread 0x7ffff7fa7740 (LWP 33923)]
Thread 1 "app" hit Breakpoint 4, do_fork_demo () at test.c:27
27          pid_t pid = fork();

(gdb)
Continuing.
[Detaching after fork from child process 34091]
[Parent Process] Spawned child PID: 34091
[Child Process] PID: 34091
Writing to pointer...
Thread 1 "app" received signal SIGSEGV, Segmentation fault.
0x00005555555552f7 in cause_crash (ptr=0x0) at test.c:23
23          *ptr = 42; // Segmentation Fault if ptr == NULL

(gdb) bt
#0  0x00005555555552f7 in cause_crash (ptr=0x0) at test.c:23
#1  0x000055555555541b in main () at test.c:54

(gdb) frame 0
#0  0x00005555555552f7 in cause_crash (ptr=0x0) at test.c:23
23          *ptr = 42; // Segmentation Fault if ptr == NULL
```

---

### Core dumped

gdb ./app core

```
(gdb) bt
#0  0x000055e9c9d1b2f7 in cause_crash (ptr=0x0) at test.c:25
#1  0x000055e9c9d1b41b in main () at test.c:56
(gdb) p ptr
$1 = (int *) 0x0
(gdb) frame 1
#1  0x000055e9c9d1b41b in main () at test.c:56
56          cause_crash(bad_ptr);
(gdb) p bad_ptr
$2 = (int *) 0x0
```

# 2. Strace

Used to trace system calls (syscalls) and signals made and received by a process when interacting with the Linux Kernel.

| Flag | Description |
| --- | --- |
| `-e trace=<syscall>` | Filters and displays only the specified system calls (e.g., `-e trace=open,read,mmap`). |
| `-f` | Traces child processes created by `fork()`, `vfork()`, or `clone()`. It also follows threads created with `clone()` when applicable. |
| `-p <PID>` | Attaches `strace` to an already running process specified by its Process ID (`PID`). |
| `-o file.txt` | Redirects the trace output to the specified file instead of printing it to `stderr`. |

---

**Files & Processes**

Objective: Understand how an application opens files, reads data, and creates child processes (fork/execve).

```
int main() {
    FILE *f = fopen("demo.txt", "w");
    if (f) {
        fputs("Hello strace!\n", f);
        fclose(f);
    }

    if (fork() == 0) {
        // child
        char *args[] = {"/bin/echo", "Child process running", NULL};
        execve(args[0], args, NULL);
    }
    // parent
    wait(NULL);
    return 0;
}
```

**Output:**

strace -f -e trace=openat,write,fork,execve ./app

```
// OS executes the binary file ./app. System call execve returns 0 indicating program loaded successfully
execve("./app", ["./app"], 0x7ffd7509bfb8 /* 79 vars */) = 0

//The dynamic linker finds and loads the standard library libc.so.6 so that the program can use standard C functions such as fopen, fputs, and printf.
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

//The dynamic linker finds and loads the standard library libc.so.6 so that the program can use standard C functions such as fopen, fputs, and printf.
openat(AT_FDCWD, "demo.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3

//write(3, ...) writes 14 bytes of string "Hello strace!" to file descriptor 3.
write(3, "Hello strace!\n", 14)         = 14

Forking Child Process (PID 39490)
strace: Process 39490 attached

//The child process overwrites its own source code with the /bin/echo program
[pid 39490] execve("/bin/echo", ["/bin/echo", "Child process running"], NULL) = 0

// /bin/echo is an independent program, so it also reloads the libc library for itself.
[pid 39490] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 39490] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

// /bin/echo executes the write system call to print the string to stdout (file descriptor 1).
[pid 39490] write(1, "Child process running\n", 22Child process running
) = 22

//child process exits with exit code 0
[pid 39490] +++ exited with 0 +++

//When the child process exits, the Linux kernel sends a SIGCHLD signal to the parent process. The wait() function in the parent process catches this signal and cleans up the child process resources.
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=39490, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---

//The parent process exits successfully (exited with 0).
```

```
+++ exited with 0 +++
```

---

### Permissions & Errors

Handle "Why is my program failing?" scenarios when encountering permission errors or missing file errors (ENOENT, EACCES).

```
int main() {
    FILE *f = fopen("/etc/shadow", "r");
    if (!f) {
        perror("Cannot open file");
    } else {
        fclose(f);
    }
    return 0;
}
```

**Output:**

strace -e trace=openat ./app

```
// The dynamic linker finds and loads the standard library libc.so.6 so that the program can use standard C functions such as fopen, fputs, perror and printf.
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

// EACCES (Permission denied). The reason is that the /etc/shadow file contains system user password hashes — by default, only the root account has read permissions. Regular users will be denied access by the Linux Kernel.
openat(AT_FDCWD, "/etc/shadow", O_RDONLY) = -1 EACCES (Permission denied)
Cannot open file: Permission denied

+++ exited with 0 +++
```

### Signals

Objective: Observe how a program catches or is terminated by signals (SIGINT, SIGTERM, SIGSEGV).

```
// Signal-safe global flag variable
volatile sig_atomic_t keep_running = 1;
```

```
void handle_sigint(int sig) {
    // Only set the interrupt flag, DO NOT call I/O or lock functions here
    keep_running = 0;
}

int main() {
    struct sigaction sa;
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    printf("Running... Press Ctrl+C to stop.\n");

    while (keep_running) {
        // Simulate running work, continuously check the flag
        sleep(1);
    }

    printf("\nCaught SIGINT (Ctrl+C)! Exiting gracefully...\n");
    return 0;
}
```

**Output:**

trace -e trace=rt_sigaction,nanosleep,%signal ./app

```
//Registering a Signal Handler (signal(SIGINT, handle_sigint))
rt_sigaction(SIGINT, {sa_handler=0x5683f21c61c9, sa_mask=[], sa_flags=SA_RESTORER,
sa_restorer=0x757b4c645330}, NULL, 8) = 0
Running... Press Ctrl+C to stop.

//Process receives the SIGINT signal sent by the kill command (PID 16762) from a second
terminal.
--- SIGINT {si_signo=SIGINT, si_code=SI_USER, si_pid=16762, si_uid=1000} ---
rt_sigreturn({mask=[]})                 = -1 EINTR (Interrupted system call)

Caught SIGINT (Ctrl+C)! Exiting gracefully...
+++ exited with 0 +++
```

# 3. Valgrind

Valgrind (specifically its default tool Memcheck) is a tool on Linux for detecting dynamic memory (Heap Memory) errors.

Compile with the -g flag

```
gcc -g -O0 program.c -o program
```

Run with Valgrind

```
valgrind --leak-check=full --track-origins=yes ./program
```

| Flag | Description |
| --- | --- |
| `-O0` | **Compiler flag:** Disables compiler optimizations, making debugging easier because the generated code stays closer to the source code. |
| `--leak-check=full` | **Valgrind option:** Provides detailed information about detected memory leaks, including the size and allocation location of leaked blocks. |
| `--track-origins=yes` | **Valgrind option:** Tracks the origin of uninitialized values to help identify where the uninitialized data was created. |

### Memory Leaks

Memory allocated with malloc/calloc is not freed with free before process termination or before losing the pointer.

```
void leak_memory() {
    int *arr = (int *)malloc(10 * sizeof(int)); // Allocate 40 bytes
    arr[0] = 100;
}

int main() {
    leak_memory();
    return 0;
}
```

**Output:**

gcc -g -o app test.c valgrind --leak-check=full ./app

```
==27857== HEAP SUMMARY:
==27857==     in use at exit: 40 bytes in 1 blocks
==27857==   total heap usage: 1 allocs, 0 frees, 40 bytes allocated
==27857==
==27857== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==27857==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==27857==    by 0x10915E: leak_memory (test.c:10)
==27857==    by 0x109181: main (test.c:15)
==27857==
==27857== LEAK SUMMARY:
==27857==    definitely lost: 40 bytes in 1 blocks
==27857==    indirectly lost: 0 bytes in 0 blocks
==27857==      possibly lost: 0 bytes in 0 blocks
==27857==    still reachable: 0 bytes in 0 blocks
```

---

### Invalid Read / Invalid Write

Accessing memory beyond array bounds (Heap Buffer Overflow) or reading/writing to an inaccessible memory region.

```
int main() {
    int *arr = (int *)malloc(5 * sizeof(int)); // Array of 5 elements (index 0..4)

    // Invalid Write error: Writing to index 5 (out of allocated memory range)
    arr[5] = 250;

    // Invalid Read error: Reading from index 5
    int val = arr[5];

    free(arr);
    return 0;
}
```

### Output

```
==28663== Invalid write of size 4
==28663==    at 0x10918B: main (test.c:13)

==28663== Invalid read of size 4
```

```
==28663==    at 0x109195: main (test.c:16)
```

---

### Use-After-Free

Reusing a pointer that points to a memory region that has already been freed using free().

```
int main() {
    int *ptr = (int *)malloc(sizeof(int));
    *ptr = 42;

    free(ptr); // Free allocated memory

    // Use-After-Free error: Still reading value after free
    printf("Value: %d\n", *ptr);

    return 0;
}
```

### Output

```
==29335== Invalid read of size 4
==29335==    at 0x1091BD: main (test.c:16)
==29335==  Address 0x4a81040 is 0 bytes inside a block of size 4 free'd
==29335==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==29335==    by 0x1091B8: main (test.c:13)
```

---

### Uninitialized Memory

Using the value of an uninitialized local variable (garbage on stack) or uninitialized malloc memory inside a conditional statement (if/else) or an I/O function.

```
int main() {
    int x; // Variable x is uninitialized

    // Error: Branching based on an uninitialized variable
    if (x == 10) {
        printf("Equal\n");
    } else {
        printf("Not equal\n");
```

```
    }

    return 0;
}
```

**Output**

gcc -g test.c -o app valgrind --track-origins=yes ./app

```
==30548== Conditional jump or move depends on uninitialised value(s)

==30548==    at 0x109159: main (test.c:13)

==30548==  Uninitialised value was created by a stack allocation

==30548==    at 0x109149: main (test.c:9)
```

# LAB / STUDIO

## LAB 1

Compile a program with a deliberately introduced segfault; debug it live with GDB (breakpoints/backtrace) and then again by loading a generated core dump

1. GBD

## LAB 2

Debug a multi-threaded program with GDB (thread apply all bt) to locate a race-condition-induced crash

```
typedef struct {
    int id;
    char data[100];
} DataNode;

DataNode *global_node = NULL;

void* worker_writer(void* arg) {
    for (int i = 0; i < 100; i++) {
        if (global_node != NULL) {
            usleep(1);
```

```
            // Race condition: Another thread can free(global_node)
            // right after this NULL check!
            global_node->id = i;
        }
    }
    return NULL;
}

void* worker_deleter(void* arg) {
    //usleep(100); // Wait a bit for the writer thread to start running
    if (global_node != NULL) {
        DataNode *tmp = global_node;
        global_node = NULL; // Set to NULL
        free(tmp);         // Free allocated memory
    }
    return NULL;
}

int main() {
    global_node = (DataNode*)malloc(sizeof(DataNode));

    pthread_t t1, t2, t3;

    // Create threads that concurrently write to and free shared memory
    pthread_create(&t1, NULL, worker_writer, NULL);
    pthread_create(&t2, NULL, worker_writer, NULL);
    pthread_create(&t3, NULL, worker_deleter, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    return 0;
}
```

**Output**

gcc -g -pthread test.c -o app

gdb ./app

```
[New Thread 0x7ffff7bff6c0 (LWP 37090)]
[New Thread 0x7ffff73fe6c0 (LWP 37091)]
[New Thread 0x7ffff6bfd6c0 (LWP 37092)]
[Thread 0x7ffff6bfd6c0 (LWP 37092) exited]

Thread 3 "app" received signal SIGSEGV, Segmentation fault.
[Switching to Thread 0x7ffff73fe6c0 (LWP 37091)]
0x0000555555555222 in worker_writer (arg=0x0) at test.c:20
20                  global_node->id = i;
(gdb) info threads
  Id   Target Id                                  Frame
  1    Thread 0x7ffff7fa7740 (LWP 37076) "app" 0x00007ffff7c98e51 in
__futex_abstimed_wait_common64 (private=128, cancel=true, abstime=0x0, op=265, expected=37090,

    futex_word=0x7ffff7bff990) at ./nptl/futex-internal.c:57
  2    Thread 0x7ffff7bff6c0 (LWP 37090) "app" 0x0000555555555222 in worker_writer (arg=0x0)
at test.c:20
* 3    Thread 0x7ffff73fe6c0 (LWP 37091) "app" 0x0000555555555222 in worker_writer (arg=0x0)
at test.c:20
(gdb) thread apply all bt

Thread 3 (Thread 0x7ffff73fe6c0 (LWP 37091) "app"):
#0  0x0000555555555222 in worker_writer (arg=0x0) at test.c:20
#1  0x00007ffff7c9cb84 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#2  0x00007ffff7d29ecc in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78

Thread 2 (Thread 0x7ffff7bff6c0 (LWP 37090) "app"):
#0  0x0000555555555222 in worker_writer (arg=0x0) at test.c:20
#1  0x00007ffff7c9cb84 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#2  0x00007ffff7d29ecc in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78

--Type <RET> for more, q to quit, c to continue without paging--
Thread 1 (Thread 0x7ffff7fa7740 (LWP 37076) "app"):
#0  0x00007ffff7c98e51 in __futex_abstimed_wait_common64 (private=128, cancel=true,
abstime=0x0, op=265, expected=37090, futex_word=0x7ffff7bff990) at ./nptl/futex-internal.c:57
#1  __futex_abstimed_wait_common (cancel=true, private=128, abstime=0x0, clockid=0,
expected=37090, futex_word=0x7ffff7bff990) at ./nptl/futex-internal.c:87
#2  __GI___futex_abstimed_wait_cancelable64 (futex_word=futex_word@entry=0x7ffff7bff990,
expected=37090, clockid=clockid@entry=0, abstime=abstime@entry=0x0, private=private@entry=128)
at ./nptl/futex-internal.c:139
```

```
#3  0x00007ffff7c9e883 in __pthread_clockjoin_ex (threadid=140737349940928, thread_return=0x0,
clockid=0, abstime=0x0, block=<optimized out>) at ./nptl/pthread_join_common.c:102
#4  0x000055555555530e in main () at test.c:46
(gdb) print global_node
$1 = (DataNode *) 0x0
(gdb) print i
$2 = 4
(gdb) thread 2
[Switching to thread 2 (Thread 0x7ffff7bff6c0 (LWP 37090))]
#0  0x0000555555555222 in worker_writer (arg=0x0) at test.c:20
20                  global_node->id = i;
(gdb) print i
$3 = 8
```

**Explain**

```
Thread 3 (* - current thread): LWP 37091, executing worker_writer function and crashed at line
20 (global_node->id = i;).
Thread 2: LWP 37090, also stopped at line 20 of worker_writer function.
Thread 1 (main): Currently in __futex_abstimed_wait_common64 state — meaning main is waiting
for threads to finish via pthread_join().
Thread 4 (LWP 37092 - worker_deleter): Exited. The initial notification line: [Thread
0x7ffff6bfd6c0 (LWP 37092) exited].

Thread 3 crashed at line 20 because worker_deleter (Thread 4) freed global_node before Thread
3 executed the write operation.
```

## LAB 3

Run strace on a program that hangs or fails silently; use the syscall trace (files/sockets/processes/permissions/signals) to find the root cause

2. strace

## LAB 4

Run Valgrind (memcheck) on a program with a deliberately introduced memory leak and a use-after-free bug; interpret the report and fix both

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct {
    int id;
    int score;
} Player;

void use_after_free_demo() {
    Player *p = (Player *)malloc(sizeof(Player));
    p->id = 101;
    p->score = 50;

    free(p); // Memory is deallocated here

    // Bug 1: Use-After-Free (reading and writing to freed memory)
    printf("Player ID: %d\n", p->id);
    p->score = 100;
}

void memory_leak_demo() {
    // Bug 2: Memory Leak (allocated 400 bytes on heap, never freed)
    int *numbers = (int *)malloc(100 * sizeof(int));
    for (int i = 0; i < 100; i++) {
        numbers[i] = i;
    }
    // Missing free(numbers);
}

int main() {
    printf("=== Running Buggy Program ===\n");
    use_after_free_demo();
    memory_leak_demo();
    printf("=== Execution Complete ===\n");
    return 0;
}
```

**Output**

```
Use-After-Free (Invalid read & Invalid write)

```text
```
```

```
==7726== Invalid read of size 4
==7726==    at 0x1091E8: use_after_free_demo (test.c:17)
==7726==    by 0x109278: main (test.c:32)
==7726==  Address 0x4a81480 is 0 bytes inside a block of size 8 free'd

==7726== Invalid write of size 4
==7726==    at 0x109204: use_after_free_demo (test.c:18)
==7726==    by 0x109278: main (test.c:32)
==7726==  Address 0x4a81484 is 4 bytes inside a block of size 8 free'd
```

Memory Leak (definitely lost)

```
in use at exit: 400 bytes in 1 blocks
400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==7726==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==7726==    by 0x109223: memory_leak_demo (test.c:23)
==7726==    by 0x109282: main (test.c:33)
```

**Fix code**

```
typedef struct {
    int id;
    int score;
} Player;

void use_after_free_demo() {
    Player *p = (Player *)malloc(sizeof(Player));
    if (p == NULL) return;

    p->id = 101;
    p->score = 50;

    // Use values BEFORE freeing
    printf("Player ID: %d\n", p->id);
    p->score = 100;

    // Fix 1: Free at the end of lifecycle and nullify pointer
    free(p);
    p = NULL;
}
```

```
void memory_leak_demo() {
    int *numbers = (int *)malloc(100 * sizeof(int));
    if (numbers == NULL) return;

    for (int i = 0; i < 100; i++) {
        numbers[i] = i;
    }

    // Fix 2: Free allocated memory before returning
    free(numbers);
    numbers = NULL;
}

int main() {
    printf("=== Running Fixed Program ===\n");
    use_after_free_demo();
    memory_leak_demo();
    printf("=== Execution Complete ===\n");
    return 0;
}
```

---

Revision #2
Created 2026-09-21 09:29:44 UTC by Dat Nguyen
Updated 2026-09-23 02:01:08 UTC by Dat Nguyen